

Delta-Sigma ($\Delta\Sigma$) Modulator

Simulation Framework – MATLAB & Simulink

Project Overview

This repository contains MATLAB scripts and Simulink models used to simulate **ideal**, **non-ideal**, and **noise-injected** Delta-Sigma ($\Delta\Sigma$) ADC architectures.

It includes a comprehensive workflow for:

- Digital decimation filter design.
- Advanced noise modeling (Flicker, Thermal, Jitter).
- ENOB (Effective Number of Bits) vs. Sampling-Rate tradeoff analysis.

System Requirements

To ensure the simulation runs correctly and within a reasonable timeframe, the following environment is required:

- **Processor:** Intel Core i7 (or equivalent high-performance CPU)
- **Operating System:** Windows 11
- **MATLAB Version:** R2025b

Note: The simulation involves complex noise integration and typically requires a minimum of 120 seconds of simulation time, which is computationally intensive.

Prerequisites: MATLAB Toolboxes

Ensure the following toolboxes are installed in your MATLAB environment:

- Mixed-Signal Blockset
 - DSP System Toolbox
 - Sigma-Delta Toolbox
 - Filter Design Toolbox
-

Repository Structure

Updated Repository Structure

MATLAB_SIMULINK :

DIGITAL_FILTER_FLOATING_POINT_REFERENCE

|-Compensation_coefficients.m

|-Halfband_coefficeint.m

FILTER_SIMULINKMODELS

|-CIC.slx

|-COMPLETE_FILTER.slx

|-FIR.slx

|-HALFBAND_FIR.slx

MODULATOR_Model_SIMULINK

CT

|-ct_ciff2_ideal.slx

|-ct_ciff3_ideal.slx

|-ct_ciff4_ideal.slx

DT

|-dt_cifb2_ideal.slx

|-dt_cifb3_ideal.slx

|-dt_ciff2_ideal.slx

|-dt_ciff3_ideal.slx

|-dt_crfb2_ideal.slx

|-dt_crfb3_ideal.slx

|-dt_crff2_ideal.slx

|-dt_crff3_ideal.slx

hybrid_2_ideal.slx

mash_ideal.slx

FinalModel :

|-dt_crfb3_non_ideal.slx

MATLAB_SCRIPTS

|-slprj (cache files)

|-Enob_vs_Samplingrate.m

|-flicker.m

|-thermal_noise_params.m

|-getCIFBparams.m

Noise-Modelling

NON_IDEAL_MODUALTORS_SIMULINK

CT

|-ct_ciff2_non_ideal.slx

|-ct_ciff3_non_ideal.slx

|-ct_ciff4_non_ideal.slx

DT

```
| -dt_cifb2_non_ideal.slx  
| -dt_cifb3_non_ideal.slx  
| -dt_ciff2_non_ideal.slx  
| -dt_ciff3_non_ideal.slx  
| -dt_crfb2_non_ideal.slx  
| -dt_crfb3_non_ideal.slx  
| -dt_crff2_non_ideal.slx  
| -dt_crff3_non_ideal.slx
```

```
hybrid_2_non_ideal.slx  
mash_non_ideal.slx  
|-noise_modelsimulink
```

OISE_MODELS_SIMULINK

```
| -clock_jitter.slx  
| -CT_NON_IDEAL_INTEGRATOR.slx  
| -DT_NON_IDEAL_INTEGRATOR.slx  
| -flicker.slx  
| -thermal_noise.slx
```

Important: The Final Modulator

The primary model chosen for evaluation is located in:

FinalModel/dt_crfb3_non_ideal.slx

This is a ****Discrete-Time CRFB (Cascade of Resonators with FeedBack)**** 3rd-order modulator. It incorporates the following non-idealities (controlled via internal switches):

- **Flicker Noise**
 - **Thermal Noise**
 - **Clock Jitter**
 - **Op-Amp Non-idealities:** Finite gain error (α), Slew rate, GBW, and Sampling period (T_s).
-

Simulation Workflow

Follow these steps strictly to reproduce the results.

Step 1: Generate Digital Filter Coefficients

Navigate to DIGITAL_FILTER_FLOATING_POINT_REFERENCE/ and run the following scripts to load coefficients into the workspace: 1. Compensation_coefficients.m 2. Halfband_coefficeint.m

Step 2: Initialize Noise Models

A. Flicker Noise

Run MATLAB_SCRIPTS/flicker.m.

- Configures F_s and f_{corner} .
- Generates the state-space matrices for the Simulink 1/f filter block.

B. Thermal Noise

Run MATLAB_SCRIPTS/thermal_noise_params.m.

- Inputs: Order, OSR, Target SNR, Temperature, Sampling Freq.
- Outputs: Sampling capacitor $C_s = \frac{kT}{P_{noise}}$, Ideal SQNR, and Noise Power.

Step 3: Run the Simulation (ENOB Analysis)

To analyze the performance: 1. Open MATLAB_SCRIPTS/Enob_vs_Samplingrate.m. 2. Ensure the script points to the correct model file (e.g., dt_crfb3_non_ideal.slx). 3. Run the script.

The script will:

- Compute decimation factors and CIC scaling.
- Run the Simulink model for the specified duration (Default: num2str(Sim_Time)).
- Extract adc_ac_out.
- Calculate SNR and ENOB.

Results will be stored in the OUT struct (e.g., OUT.out_0p5k, OUT.out_1k).

Critical Notes

1. **Simulation Duration:** The minimum simulation time must be set to **120 seconds** to obtain valid SNR readings from the ADC-AC Measurement block.
2. **Output:** All numerical results (SNR/ENOB/SQNR) are printed directly to the MATLAB Command Window.

Non-Ideality Modelling Blocks inside the modulators architecture of each non ideal file

The following figures illustrate the Simulink implementations of each non-ideality used in the $\Delta\Sigma$ modulator.

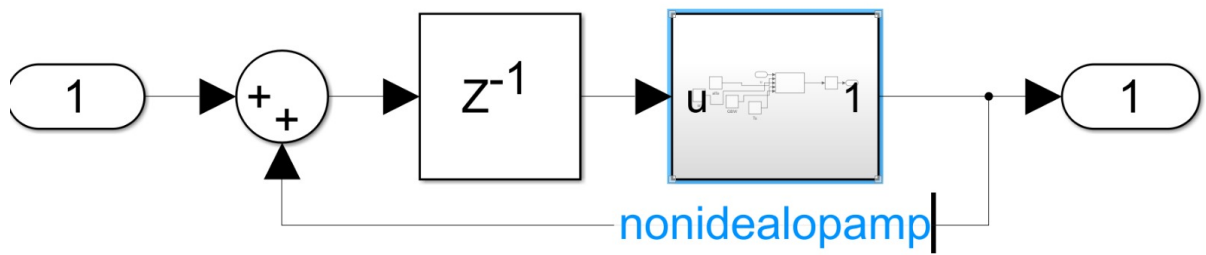


Figure 1: Non-Ideal Integrator Block (DT/CT Implementation)

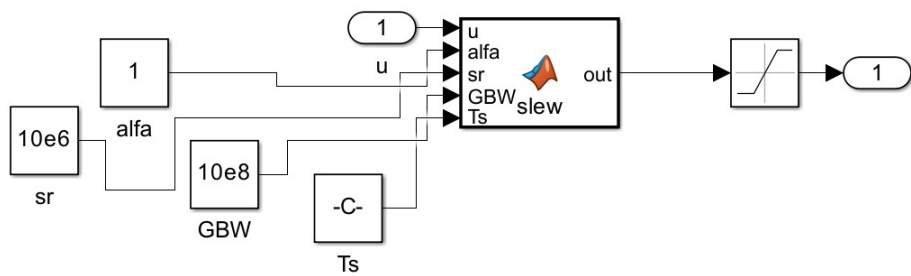


Figure 2: Alternate Op-Amp Non-Ideal Model (Internal Error Modelling)

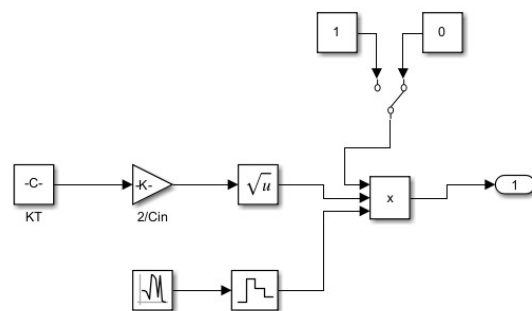


Figure 3: Thermal Noise Modelling Block

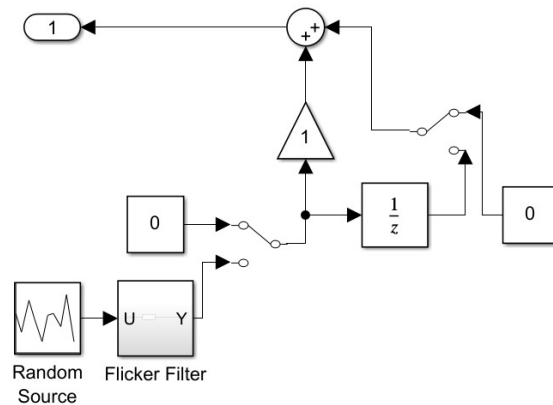


Figure 4: Flicker Noise ($1/f$) Modelling Block

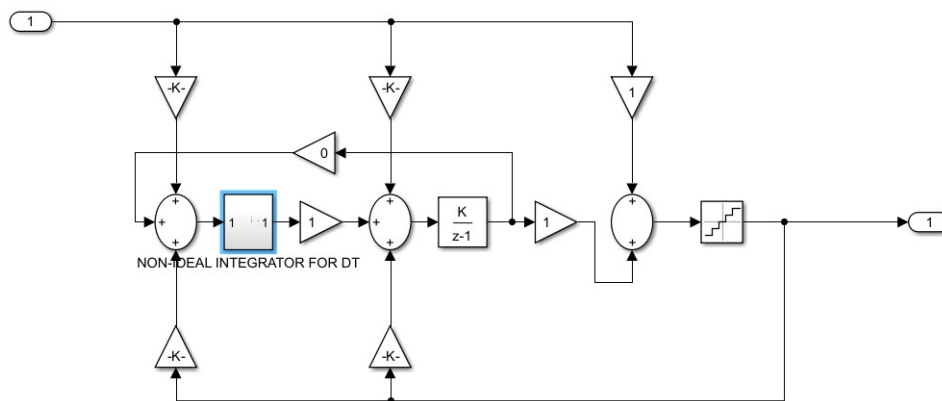


Figure 5: Complete Non-Ideal $\Delta\Sigma$ Modulator Architecture

How SNR is Computed in This Framework

The Signal-to-Noise Ratio (SNR) evaluation is performed using the **ADC AC Measurement** block from the **Mixed-Signal Blockset**. This block internally uses a highly optimized FFT-based method to ensure accurate AC performance characterization of Delta-Sigma ADCs.

Why the ADC AC Block is Used

- MATLAB provides an optimized FFT engine that minimizes numerical leakage.
- The block automatically enforces **coherent sampling**, ensuring the input sinewave aligns perfectly with an FFT bin.
- The simulation stop time is automatically adjusted based on the output sampling rate and input tone frequency to avoid spectral leakage.
- The method is more reliable than manual FFT computation because it applies appropriate

windowing, bin selection, and dynamic range scaling.

FFT Bin Method Used Internally

The ADC AC Measurement block computes SNR using the following process:

1. After decimation, the block collects a buffer of output samples.
2. Performs an N -point FFT over the captured data.
3. Identifies the **signal bin** corresponding to the input tone.
4. All other bins (except DC and harmonics) are treated as **noise**.

$$\text{SNR (dB)} = 10 \log_{10} \left(\frac{P_{\text{signal}}}{P_{\text{noise}}} \right)$$

Where:

- P_{signal} = FFT power in the tone bin.
- P_{noise} = Total FFT power excluding the tone bin, DC, and harmonics.

Coherent Sampling Enforcement

To avoid spectral leakage:

$$f_{in} = \frac{k}{N} f_s$$

Where:

- f_{in} = Input sine frequency given by the user.
- f_s = Output sampling rate after decimation.
- k = Coherent integer bin index.

The ADC AC block automatically selects N and simulation stop time such that:

$$\frac{f_{in}}{f_s} = \frac{k}{N}$$

Thus the input waveform completes an integer number of cycles inside the FFT window.

Advantages of This Method

- Zero leakage in spectral bins.
- No need to manually tune the simulation time.
- Produces stable and repeatable SNR results.
- Works reliably with long simulation times (e.g., 120 seconds).

Summary

The ADC AC Measurement block ensures an accurate, robust SNR calculation by combining:

- Coherent sampling timing,
- Optimized FFT bin processing,
- Automatic noise floor separation,
- MATLAB's internally enhanced DSP routines.

This makes it the preferred method for SNR and ENOB evaluation for your Delta-Sigma ADC architecture.

Troubleshooting: Script Not Running

If any problem occurs while running the main automation script (`Enob_vs_Samplingrate.m`)—for example, errors related to callbacks, initialization functions, model loading, or filter parameters—you can run the setup manually using the following procedure:

Manual Run Procedure

1. Choose the decimation factor manually. The decimation is always of the form:

$$\text{Decimation Factor} = 64 \times D$$

Set the multiplier:

$$M = 1$$

2. Provide the required modulator sampling frequency:

Fs_mod

3. Provide the input stimulus frequency:

inputF

4. After setting these variables in the MATLAB workspace, you can run the model manually by entering the following command in the MATLAB Command Window:

```
sim("D:\MATLAB\MATLAB_SIMULINK_MODELS\IDEAL_MODULATORS_SIMULINK\DT\dt_crfb3_ideal.slx",  
    'StopTime', num2str(Sim_Time));
```

This directly simulates the model for the duration specified by `Sim_Time`.

This bypasses the automation script and forces Simulink to simulate the model using the parameters you manually supplied.